



CPE 334 Introduction to Software Engineering in the Age of AI Agents

Sections 1, 2, HS, 31 and 32. Semester: 1/2026.

Lab 1. TokTickIT (ตอกติกิต) Full-Stack Hello World Starter.

Score: ____ / 40

Instructors:	Assoc. Prof. Suthep Madarasmi, Ph.D. (Jogie/โจ๊ก)	(suthep.mad@kmutt.ac.th)
	Aj. Piyanit Ua-areemitr, Ph.D (Toey)	(piyanit.wep@kmutt.ac.th)
	Aj. Santawat Thanyadit Ph.D. (Job)	(santawat.than@kmutt.ac.th)
TAs:	Rachawipa Katippatee (Bom)	(rachawipa.kati@gmail.com)
	Kantapat Suwannahong (Bump)	(kantapat.suwan@kmutt.ac.th)
	Rattanachote Petpansri (Loogmoo)	(rattanachote.petpa@kmutt.ac.th)
	Prapatsorn Sangrod (Noon)	(prapatsorn.sangr@kmutt.ac.th)
	Supachok Deetaweessukh (Tik)	(jedsadaporn.pann@mail.kmutt.ac.th)

Individual Sprint 1:

React UI → Express REST API → Prisma ORM → PostgreSQL DB

Sprint Goal: Build a tiny but complete vertical slice proving that every layer of the tech stack works well.

1. What We Are Building Across Labs 1 to 4

TokTickIT is an IT service desk application for Account and Access, Hardware, Software, and Network requests. Across seven individual sprints, each student will incrementally build the same product from a full-stack foundation into a polished local web application.

The instructor acts as the stakeholder and product owner, releasing a new engineering contract for each sprint. Every contract defines the required behavior, UI, business rules, acceptance criteria, and tests. Students use AI coding agents to assist with implementation, but remain responsible for the specifications, code, tests, reviews, and final product quality.

Specification: defines the product behavior and design.

Engineering contract: defines the specification plus the evidence required to prove it is complete.

The final application will support three roles: Requester, IT Staff, and Administrator. A Ticket stores the current state of the request and contains related Public Comments, Internal Notes, Actions Taken, and Attachments. Requesters and IT Staff share some functions, such as public comments and attachments, while role-based rules control sensitive actions such as assignment, IT priority, status changes, internal notes, and user management.

By the end of Lab 4, every student should have a professional application running locally with responsive screens, consistent UI styling, clear validation and warnings, safe error handling, automated tests, GitHub Issues, feature branches, peer-reviewed Pull Requests, and complete documentation. CI/CD and cloud deployment will be introduced later during the team-project phase.

1.1. User Roles and Role-Specific Actions

Each user has one user role. There are 3 user roles: Requester, IT Staff, and Administrator. The Administrator role includes all IT Staff permissions in addition to user and reference-data management permissions.

Requester, IT Staff, and Administrator can all: • view permitted ticket information • add public comments • add attachments	Requester can: • create a ticket • view their own tickets • set Requested Priority • respond to IT Staff • confirm or reject a resolution • request reopening
IT Staff and Administrator can both: • become or change the Ticket Owner • set IT Priority • change ticket status • add internal notes • add and update actions taken • resolve and close tickets	Administrator can: • view and search users • create user accounts • edit user details • assign a user role • activate or deactivate accounts • set a new initial password • manage Categories and Related Systems

1.2. Final Ticket Model

Parent entity: Ticket No. of Child entities: 5 Ticket — Public Comments — Internal Notes — Actions Taken — Attachments	The Ticket header stores the current state, including: • Ticket Number • Requester • Category • Related System • Summary and Description • Requested Priority • IT Priority • Current Status • Ticket Owner • Important Dates • Resolution Summary
---	---

1.3. Final Main Screens

- Login
- First Password Change
- Requester Dashboard
- Create Ticket
- Requester Ticket Detail
- IT Staff Dashboard
- IT Staff Ticket Detail
- User Management
- Reference Data Management
- Access Denied
- Not Found

1.4. Final Quality Expectations

The completed local application should include:

- responsive desktop, tablet, and mobile layouts
- consistent color theme
- defined styles for labels, textboxes, buttons, grids, badges, and validation
- editable and non-editable field styles
- loading, empty, success, warning, and error states
- clear field-level validation
- unit, API, UI, and E2E tests
- GitHub Issues, feature branches, Pull Requests, and peer review
- complete setup and usage documentation

1.5. Example UI Screen

This screen is provided as illustration only. For Lab 1 you are not to implement any UI yet.



Ticket No.

TKT-2025-001234

Ticket Date

May 12, 2025 09:14 AM

Category

Hardware ▾

Related System

Corporate Laptop

Requester

Jennifer Anderson

Requested Priority

Medium

IT Priority

Medium

Current Status

In Progress

Ticket Owner

Michael Brown (IT Support)

Summary

Laptop battery drains quickly

Description

My laptop battery is draining much faster than usual even when the system is idle. This started happening after last week's Windows update.

Resolution Summary

No resolution summary available yet.

Public Comments 3



Attachments 2



Service Actions 1



Event Log 6

Add Comment

[Post Comment](#)

JA

Jennifer Anderson Requester

Thank you for the update. Please let me know if you need any additional information.

May 13, 2025 11:45 AM



MB

Michael Brown IT Support

We are investigating the issue on your device. We'll update you shortly.

May 13, 2025 10:30 AM



JA

Jennifer Anderson Requester

Just adding that this issue occurs even when I close all applications.

May 12, 2025 09:20 AM



2. Lab 1 Learning Outcomes

- Build a full-stack vertical slice: React/Vite/Bootstrap UI → Express API → Prisma/PostgreSQL.
- Implement REST endpoints, automated testing (Vitest/Supertest), and Git Flow.
- Utilize GitHub Projects for task tracking and peer-reviewed Pull Requests.

3. Lab 1 Request from Stakeholder

Here the stakeholder refers to the product owner, customer, or end user requesting this application.

“Before developing features for this ticketing app, the IT department wants evidence that the proposed technology stack works as one integrated system. Create a basic TokTickIT application that displays the service status and the supported request categories stored in the database.”

3.1. Required Working Result

At the end of this lab, opening the frontend in a browser must show the app name and a [Check System] button that when clicked shows the system status and the four request categories.

- Backend status: Online, based on a real API call for health check.
- The four supported categories loaded from PostgreSQL through the backend via API call.
- A loading state (hour glass ... “loading”) while data is being requested.
- A useful error message when the backend or database is unavailable.

TokTickIT IT Service Desk

[Check System]

System Status: Online

Supported Request Categories:

- Account and Access
- Hardware
- Software
- Network

4. Technology and Scope Constraints

Area	Required choice
Frontend	React + TypeScript + Vite + Bootstrap
Backend	Node.js + Express + TypeScript
Database	PostgreSQL + Prisma
Architecture	REST-style APIs
Testing	Vitest and Supertest in Lab 1
Workflow	Git, GitHub Projects, four Issues, main/dev/feature branches, Pull Requests, peer review

Do not substitute another framework, database, ORM, or UI library. Playwright, authentication, ticket creation, and image upload will be introduced in later labs.

5. IDE Use

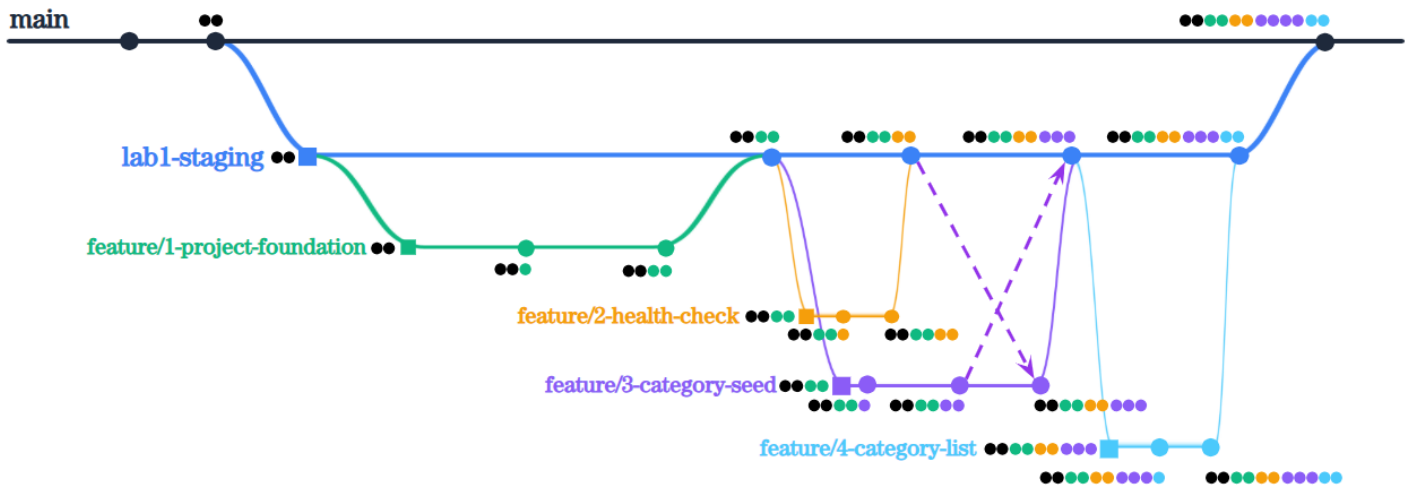
For this course, students will use a VS Code-based IDE that includes an integrated AI coding assistant, most likely Antigravity, subject to the availability of Google Cloud Platform support for the course.

Many development tasks may be completed with assistance from the IDE agent, including project setup, Git operations, coding, debugging, documentation, and preparation of detailed SDD and TDD drafts. Students are expected to use an AI agent / LLM to help make specifications, acceptance criteria, and test scenarios more complete.

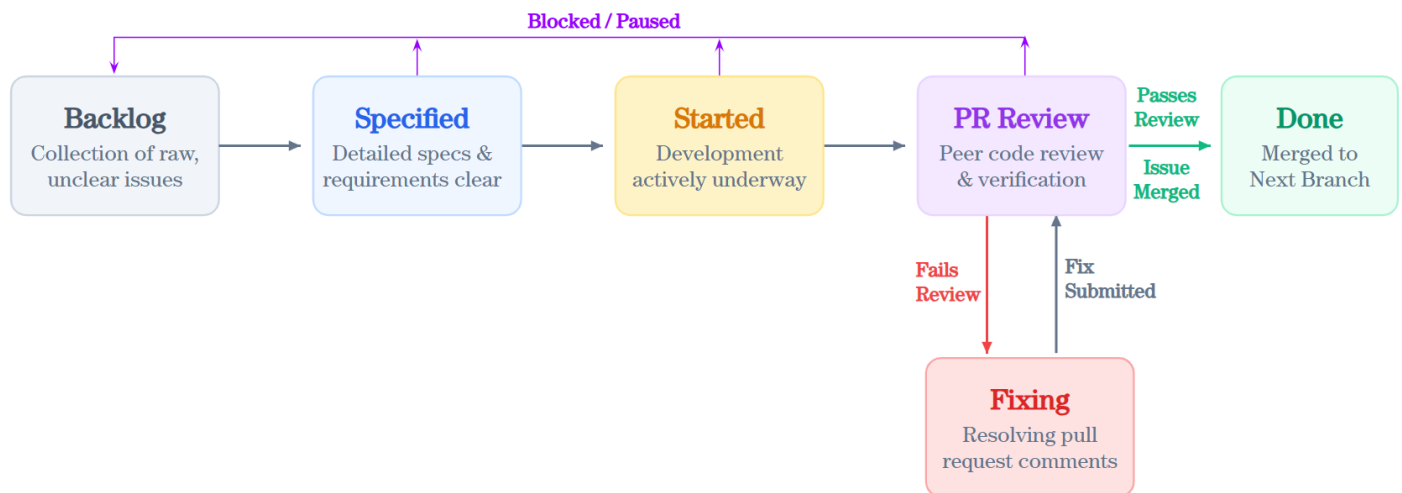
However, the AI agent is an assistant, not the owner of the work. Students must review, correct, and approve all generated specifications, tests, code, commands, and Git operations. They must understand each step of the workflow, know what they are asking the agent to do, and be able to explain the resulting work. You may delegate execution to the IDE agent, but you may not delegate understanding, judgment, or responsibility.

6. Lab 1 Execution Steps

1. **Setup:** Find a peer reviewer, create the GitHub repository, and initialize *main* and *lab1-staging* branches.
2. **Planning:** Create the GitHub Project board and the four Issues (Section 7) in the Backlog.
3. **Git Flow via the git branch structure:** Use *main* as the stable release (production) branch and *lab1-staging* as the Lab 1 integration branch. Create *lab1-staging* from *main* and push it to GitHub. Do not develop directly on *main* or *lab1-staging*.



4. **Create the GitHub Project:** Create a GitHub Project named TokTickIT Individual Sprints. Use a Board view with these statuses in this exact order: Backlog, Specified, Started, PR Review, Fixing, Done. A status moves from PR Review to Fixing if it did not pass review; otherwise, it moves to done. Issues move from Started to PR Review and also from Fixing to PR Review.



7. Required GitHub Issues and Kanban Workflow

Create all four Issues (Tasks) before implementation begins. Add every Issue to the Project with initial status *Backlog*. Move an Issue to *Specified* only after you have read and understood its requirements. Move only the Issue you are actively implementing to *Started*.

7.1. Issue 1: Set up the TokTickIT project foundation

Type: Technical setup

Required branch: feature/1-project-foundation

Acceptance criteria:

- React + TypeScript + Vite frontend starts successfully.
- Bootstrap is installed and visible in the frontend.

- Node.js + Express + TypeScript backend starts successfully.
- PostgreSQL is reachable and Prisma is initialized.
- Vitest and Supertest commands are configured.
- .gitignore and .env.example exist; secrets and node_modules are not committed.
- Initial README setup instructions are present.

7.2. Issue 2: Implement the API health check

Type: Feature

Required branch: feature/2-health-check

Acceptance criteria:

- GET /api/health returns HTTP 200.
- The JSON response contains status = ok and service = TokTickIT API.
- A Supertest test verifies the endpoint.
- The React page displays the backend status based on a real API call.
- A useful error message appears when the backend is unavailable.

7.3. Issue 3: Create and seed IT request categories

Type: Database preparation

Required branch: feature/3-category-seed

Acceptance criteria:

- A Prisma Category model exists with id, unique name, and createdAt.
- A migration creates the Category table.
- The seed inserts Account and Access, Hardware, Software, and Network.
- The seed is safe to run more than once without duplicates.
- Database credentials are not committed.

7.4. Issue 4: Display the IT request category list

Type: Feature

Required branch: feature/4-category-list

Acceptance criteria:

- GET /api/categories retrieves categories from PostgreSQL through Prisma.
- The API returns each category ID and name in a predictable order.
- A Supertest test verifies the response.

- React displays the categories returned by the API, not hard-coded values.
- Loading and error states are shown.
- A Vitest test verifies the category-list UI behavior.

Dependency order: Issue 1 must be completed first. Issues 2 and 3 may then proceed. Issue 4 starts only after Issue 3 is available in dev.

8. Required Repository Structure

```
toktickit/
├── client/
├── server/
│   ├── prisma/
│   ├── src/
│   └── tests/
│       └── lab-01/
├── docs/
│   └── lab-01/
│       ├── ai_use.md
│       └── reviewer.md
├── .gitignore
└── README.md
```

9. Minimum Database Model

```
model Category {
  id    Int    @id @default(autoincrement())
  name   String @unique
  createdAt DateTime @default(now())
}
```

10. Required REST Endpoints

10.1. Health check

GET /api/health

```
200 OK
{"status": "ok",
 "service": "TokTickIT API"
}
```

10.2. Category list

GET /api/categories

```
200 OK
[
  { "id": 1, "name": "Account and Access" },
  { "id": 2, "name": "Hardware" },
  { "id": 3, "name": "Software" },
  { "id": 4, "name": "Network" }
]
```

11. Required Automated Tests

Tool	Minimum test	Expected evidence
Supertest	GET /api/health returns 200 and status = ok	Passing terminal output
Supertest	GET /api/categories returns the seeded categories	Passing terminal output
Vitest	TokTickIT heading renders	Passing terminal output
Vitest	At least one loading, success, or error state behaves correctly	Passing terminal output

12. Workflow Summary

All work must occur on feature branches and enter main via lab1-staging. Peer review is mandatory for all PRs. Record reviewer details in docs/lab-01/reviewer.md.

Status	When to use it
Backlog	The Issue has been provided but has not yet been reviewed and understood.
Specified	The Issue is understood and ready to implement.
Started	The feature branch has been created and implementation has begun.
PR Review	A Pull Request to lab1-staging is open and the peer reviewer is checking it.
Fixing	Review changes are required or tests failed; corrections are being made on the same branch.
Done	The PR is approved, tests pass, it is merged into lab1-staging, and all acceptance criteria are satisfied.

Issue	Feature branch	Pull Request target
1. Project Foundation	feature/1-project-foundation	lab1-staging
2. API Health Check	feature/2-health-check	lab1-staging
3. Create and Seed Categories	feature/3-category-seed	lab1-staging
4. Display Category List	feature/4-category-list	lab1-staging

13. AI Coding Agent Rules

You may use an AI coding agent, but you remain responsible for every file, command, dependency, and test. Give the agent small tasks with clear constraints. Do not submit code you cannot explain.

14. Submit One PDF File

Grading a large number of lab submissions is challenging, so please submit one PDF file only and follow the instructions below carefully. You may use LLMs to assist you, but do keep the PDF concise and focused. Unnecessarily long responses may receive a penalty. To make grading faster and more consistent, **you must submit a single PDF using the following format:**

Answer Part 1:

[Place your content here]

Answer Part 2:

[Place your content here]

Answer Part 3:

[Place your content here]

Answer Part 4:

[Place your content here]

Part	Points	Required Submission Evidence
1. Git Use with Engineering Workflow	15	- URL List for your GitHub repository, GitHub Project, all GitHub Issues in your Project, all Pull Requests (feature branches, staging branch, main branch). - Evidence you used GitHub Project, showing list of issues as Kanban board, and final board all in “Done” Kanban state. - Screenshot evidence that you used the git workflow by showing your commit history in the final main branch showing you created various feature branches that were eventually merged into the staging branch and then the main branch. - Screenshot showing the entire Directory Structure of your repository in your IDE. It should align with what’s shown in section “Required Repository Structure”. These files should appear: docs/lab-01/tests.md, docs/lab-01/reviewer.md, docs/lab-01/ai_use.md, README.md, and test files under tests/lab-01/. - Rendered README.md (with content as per requirements stated above) and .gitignore content. - PR Review Evidence (5 points): - Rendered version of docs/lab-01/reviewer.md containing reviewer name, student ID, GitHub username, and reviewed PR links. - Evidence that your peer partner approved your submitted Pull Requests. What review comment did your partner give you for your Pull Request, and how did you respond? - Evidence that you reviewed and approved your partner's Pull Requests. What review comment did you give your partner for their Pull Request and how did your partner respond? - Screenshots of the board showing all four Issues in the Done board in your Kanban.

Part	Points	Required Submission Evidence		
2. Tests	10	Lab 1 tests are intended to prove that the initial TokTickIT vertical slice works correctly.		
		- Provide evidence via screenshot or copied output showing all tests passed in the main branch		
		- Print docs/lab-01/tests.md (rendered version) so it's easier to grade. All tests listed should be found in folder tests/lab-01. Example data in tests.md:		
		Test File tests/lab-01/	Tool	Test Description
		API-01	Supertest	Health endpoint returns 200 and expected JSON
		API-02	Supertest	Categories endpoint returns the four seeded categories
		UI-01	Vitest	TokTickIT heading renders
UI-02	Vitest	Loading state changes to category list		
UI-03	Vitest	API failure displays a useful error message		

3. AI Use and Reflection	5	1. AI Use with a very brief reflection. Print docs/lab-01/ai_use.md (rendered version) that mentions the LLM you used and provides a table of selected key prompts (6-10 is enough). Provide some reflection on your experience in improving the prompts. Here's an example content: I used the Antigravity coding agent through my Google Cloud Platform account. I mainly used Gemini 3.5 Flash as the LLM with a thinking level of Medium. Selected Key Prompts: <table><tr><th>Prompt Name</th><th>Actual Prompt Text</th></tr><tr><td>Plan Lab 1 Implementation</td><td> Read the enclosed TokTickIT Lab 1 requirements. Summarize the four GitHub Issues, their dependencies, required outputs, and required automated tests. Propose an implementation order, but do not write code yet. My Reflection: This worked in one shot. </td></tr><tr><td>Set Up Full-Stack Project</td><td> Setup the TokTickIT project tech stack as given in Lab 1 using React, TypeScript, Vite, and Bootstrap for the frontend, and Node.js, Express, and TypeScript for the backend. Configure PostgreSQL and Prisma. Use the required folder structure. Do not add functionality beyond the Lab 1 scope. My Reflection: I had to follow up this prompt with a few more to detail ... </td></tr><tr><td>Implement Health Check</td><td> Add GET /api/health to the existing Express backend. It must ... My Reflection: ... </td></tr><tr><td>Implement Category Feature</td><td> Create the Prisma Category model, ... My Reflection: ... </td></tr><tr><td>Build and Test Check System UI</td><td> Create a Bootstrap-based page with [Check System] button. When clicked, show a loading state, call ... My Reflection: ... </td></tr><tr><td>Review Final Lab 1 Work</td><td> Review the completed TokTickIT Lab 1 implementation against all acceptance criteria ... </td></tr></table>	Prompt Name	Actual Prompt Text	Plan Lab 1 Implementation	Read the enclosed TokTickIT Lab 1 requirements. Summarize the four GitHub Issues, their dependencies, required outputs, and required automated tests. Propose an implementation order, but do not write code yet. My Reflection: This worked in one shot.	Set Up Full-Stack Project	Setup the TokTickIT project tech stack as given in Lab 1 using React, TypeScript, Vite, and Bootstrap for the frontend, and Node.js, Express, and TypeScript for the backend. Configure PostgreSQL and Prisma. Use the required folder structure. Do not add functionality beyond the Lab 1 scope. My Reflection: I had to follow up this prompt with a few more to detail ...	Implement Health Check	Add GET /api/health to the existing Express backend. It must ... My Reflection: ...	Implement Category Feature	Create the Prisma Category model, ... My Reflection: ...	Build and Test Check System UI	Create a Bootstrap-based page with [Check System] button. When clicked, show a loading state, call ... My Reflection: ...	Review Final Lab 1 Work	Review the completed TokTickIT Lab 1 implementation against all acceptance criteria ...
Prompt Name	Actual Prompt Text															
Plan Lab 1 Implementation	Read the enclosed TokTickIT Lab 1 requirements. Summarize the four GitHub Issues, their dependencies, required outputs, and required automated tests. Propose an implementation order, but do not write code yet. My Reflection: This worked in one shot.															
Set Up Full-Stack Project	Setup the TokTickIT project tech stack as given in Lab 1 using React, TypeScript, Vite, and Bootstrap for the frontend, and Node.js, Express, and TypeScript for the backend. Configure PostgreSQL and Prisma. Use the required folder structure. Do not add functionality beyond the Lab 1 scope. My Reflection: I had to follow up this prompt with a few more to detail ...															
Implement Health Check	Add GET /api/health to the existing Express backend. It must ... My Reflection: ...															
Implement Category Feature	Create the Prisma Category model, ... My Reflection: ...															
Build and Test Check System UI	Create a Bootstrap-based page with [Check System] button. When clicked, show a loading state, call ... My Reflection: ...															
Review Final Lab 1 Work	Review the completed TokTickIT Lab 1 implementation against all acceptance criteria ...															

Part	Points	Required Submission Evidence
		My Reflection: ...
4. App Demo	10	Screenshot of the running browser page with app demo to show that the two http requests are working. When the app comes on, include a [Check System] button with behavior shown below. TokTickIT IT Service Desk [Check System] Success Case after click [Check System]: TokTickIT IT Service Desk [Check System] System Status: Online Supported Request Categories 1. Account and Access 2. Hardware 3. Software 4. Network Failure Case after click [Check System] (eg., when DB server not started): TokTickIT IT Service Desk [Check System] System Status: Offline Unable to connect to TokTickIT API